

UNIVERSIDAD DIEGO PORTALES



FACULTAD DE INGENIERÍA Y CIENCIAS

TAREA 2: ANÁLISIS DEL PROTOCOLO XMPP

Estudiantes:
Renato Riveros
Cristobal Gonzalez

Introducción

El presente documento detalla el análisis práctico y teórico del protocolo XMPP (eXtensible Messaging and Presence Protocol) utilizando la herramienta de captura de paquetes Wireshark. El objetivo principal es comprender la estructura de este protocolo, observar su comportamiento en una red controlada mediante contenedores y evaluar las posibles consecuencias de la manipulación de su tráfico. El informe se estructura en dos partes fundamentales requeridas para esta entrega: un marco teórico descriptivo y un análisis del tráfico capturado junto a modificaciones

Desarrollo

1. Marco Teórico

Descripción del Protocolo

XMPP es un protocolo de comunicaciones abierto y descentralizado basado en XML (Extensible Markup Language). A diferencia de otros protocolos de red que utilizan formatos binarios, XMPP intercambia datos en un formato de texto estructurado y legible. Funciona bajo una arquitectura cliente-servidor, permitiendo que las entidades se comuniquen de forma casi instantánea. Fue estandarizado por la IETF y es ampliamente reconocido por su extensibilidad.

Principal Objetivo

El objetivo central de XMPP es facilitar la mensajería instantánea, el intercambio de información de presencia (disponibilidad de los usuarios) y la gestión de listas de contactos en tiempo real. Gracias a su naturaleza extensible, su uso se ha expandido a otras áreas como el Internet de las Cosas (IoT), sistemas de señalización para WebRTC y juegos en línea.

Descripción del Tráfico Esperado

Al monitorear una sesión de XMPP en Wireshark, el tráfico esperado sigue un flujo secuencial:

- **Handshake TCP:** Establecimiento de la conexión estándar (SYN, SYN-ACK, ACK) típicamente sobre el puerto 5222 para clientes.
- **Inicio de Stream XML:** Se transfiere la etiqueta base `<stream:stream>` en texto plano para iniciar la comunicación.
- **Negociación TLS:** Intercambio de mensajes STARTTLS para asegurar la conexión. Posterior a esto, el tráfico se cifra.
- **Autenticación SASL:** Negociación de credenciales del usuario.
- **Intercambio de Stanzas XML:** Flujo de etiquetas funcionales como `<message>`, `<presence>` e `<iq>`.

2. Instalación de servicios

Para el desarrollo de la experiencia, se levantó una infraestructura virtualizada mediante contenedores Docker. Se desplegó un servidor XMPP (Ejabberd) y se configuró un cliente (Profanity).

Infraestructura de Red Aislada

Para permitir la resolución de nombres interna y la comunicación directa entre contenedores sin exponer puertos al tráfico público innecesario, se procedió a crear una red virtual puente (bridge) en Docker con "docker network create red-xmpp".

Contenedorización del Cliente

Para el repositorio de Profanity se diseñó un Dockerfile a medida basado en una imagen limpia. Este archivo automatiza la inyección de dependencias de desarrollo y la resolución de subproyectos en el aire (fallback) para la librería de protocolo libstrophe.

```
FROM ubuntu:22.04
ENV DEBIAN_FRONTEND=noninteractive

RUN apt-get update && apt-get install -y \
    meson \
    ninja-build \
    pkg-config \
    libtool \
    autoconf \
    automake \
    autoconf-archive \
    libglib2.0-dev \
    libreadline-dev \
    libncursesw5-dev \
    libcurl4-openssl-dev \
    libexpat1-dev \
    libssl-dev \
    uuid-dev \
    libgpgme-dev \
    libotr5-dev \
    libsignal-protocol-c-dev \
    libgcrypt20-dev \
    libsqlite3-dev \
    git \
    && rm -rf /var/lib/apt/lists/*

WORKDIR /app

COPY . .

RUN meson setup build --wrap-mode=forcefallback && ninja -C build install

CMD ["profanity"]
```

Figura 1: Archivo Dockerfile

Utilizando el contexto local del repositorio clonado, se generó la imagen personalizada bajo la etiqueta taller-redes-profanity con "docker build --no-cache -t taller-redes-profanity .".

Despliegue del Servidor

Para el servidor de mensajería se optó por la imagen empresarial oficial ejabberd/ecs. El despliegue se realizó inyectando variables de entorno específicas para forzar la inicialización del Host Virtual en el dominio localhost, con el siguiente comando de despliegue.

```
docker run --name mi-servidor-xmpp \
  --network red-xmpp \
  -d \
  -p 5222:5222 \
  -p 5269:5269 \
  -p 5280:5280 \
  -e EJABBERD_ADMIN_USER="admin" \
  -e EJABBERD_ADMIN_PASSWORD="password123" \
  -e EJABBERD_HOSTS="localhost" \
  ejabberd/ecs
```

Figura 2: Comando de despliegue del servidor XMPP

3. Configuración de servicios

Aprovisionamiento de Usuarios en el Servidor

Para simular un entorno de red real e interactivo, se interactuó con la herramienta de control del servidor ejabberdctl mediante la ejecución de comandos remotos en el contenedor para registrar dos identidades de prueba en el dominio localhost con "docker exec -it mi-servidor-xmpp ejabberdctl register tobal localhost clave123 y "docker exec -it mi-servidor-xmpp ejabberdctl register redes localhost clave123"

Enrutamiento del Cliente a la Red Virtual

Para que el contenedor del cliente previamente compilado pudiese resolver la dirección IP privada del servidor, se acopló dinámicamente a la red con "docker network connect red-xmpp mi-cliente-xmpp".

4. Uso y Operación del Entorno

Inicialización de los Clientes Interactivos

Se levantaron dos instancias independientes de Profanity mapeadas a sus respectivas terminales (TTY) con "docker start -ai mi-cliente-xmpp"(comando ejecutado en las dos terminales) abriendo la terminal de Profanity.

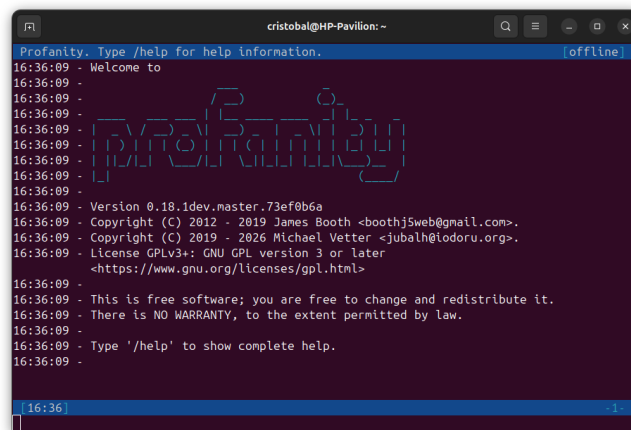


Figura 3: Terminal cliente Profanity

Negociación de Conexión en Profanity

Aunque esto corre sobre un ambiente controlado sin certificados firmados, la conexión se negoció igualmente con el cifrado de capa de transporte obligatorio, apuntando al puente Docker mi-servidor-xmpp con el comando /connect tobal@localhost server mi-servidor-xmpp.^{en} la terminal de Profanity.

Establecimiento de Presencia e Intercambio de Mensajería

Una vez pasados los procesos de autenticación, las entidades agregaron recíprocamente sus estados de presencia al listado de contactos (Roster) y procedieron al envío de cargas útiles de datos (Payloads) en formato XML con `/roster add redes@localhost` para agregar otro cliente y con `/msg redes@localhost "Mensaje de prueba de tráfico XMPP` para enviar mensajes una vez estén los dos agregados entre ellos.

Este flujo operativo garantizó la generación de tráfico limpio en el puerto 5222, quedando disponible para el posterior análisis espectral y estructural de los paquetes a través de la interfaz de red interna mapeada en Wireshark.

5. Análisis de Tráfico

El proceso de comunicación fue generado conectando el cliente al servidor contenedorizado e intercambiando mensajes de prueba, los cuales fueron capturados a través de las interfaces virtuales.

Tráfico Observado

A través de Wireshark, se aisló el tráfico correspondiente al puerto 5222, el cual es el puerto estándar asignado por la IANA para la comunicación cliente-servidor en XMPP. El análisis de las tramas capturadas revela de forma clara las tres fases fundamentales del ciclo de vida del protocolo: el establecimiento de la capa de transporte, la negociación del flujo de datos seguro y el intercambio persistente de cargas útiles cifradas.

Fase 1: Establecimiento del Canal de Transporte (TCP Handshake)

En la figura 4 se observa el inicio de la sesión del Cliente 1 (172.19.0.3) hacia el servidor (172.19.0.2). En la trama número 8 se evidencia el segmento clásico [SYN, ACK] enviado por el servidor al puerto efímero 51052 del cliente, respondiendo al SYN inicial. La trama 11 completa el acuerdo de tres vías (Three-Way Handshake) mediante un paquete [ACK] con un tamaño de ventana de recepción configurado en `Win=65536`. Este paso es crítico, ya que XMPP requiere un protocolo orientado a la conexión y libre de errores en la capa de transporte antes de instanciar sus flujos de datos.

Fase 2: Apertura de Flujos e Inicio de TLS

Inmediatamente después de estabilizarse la sesión TCP, se aprecia la naturaleza nativa del protocolo basada en XML. En la trama 12, el cliente inicia el flujo enviando la etiqueta inicial `STREAM <localhost`. En las tramas subsiguientes (14 y 17), el servidor responde anunciando sus capacidades a través de la trama `FEATURES` y el comando `PROCEED`.

Dado que se configuró el cliente para permitir la seguridad de la capa de transporte (`/tls allow`), el servidor fuerza una transición de texto plano a texto cifrado bajo el estándar TLSv1.3. En la trama 19 se observa el paquete compuesto `Server Hello, Change Cipher Spec`, donde el servidor negocia los algoritmos criptográficos y entrega sus parámetros de intercambio de claves. A partir de la trama 22, todas las tramas de control de XMPP dejan de ser legibles en texto plano y pasan a encapsularse como `Application Data`.

Fase 3: Intercambio Concurrentes de Mensajería y Análisis de Métricas de Red

La figura 5 de tráfico ilustra el comportamiento de la red cuando ambos clientes están en sesión activa. Se observa la incorporación del Cliente 2 (172.19.0.4), el cual genera un flujo de datos paralelo hacia el servidor en el mismo puerto 5222. Cada interacción o mensaje de texto plano enviado desde las consolas interactivos de Profanity genera múltiples paquetes TLS de tipo `Application Data` de longitud variable (típicamente entre 114 y 198 bytes), lo que refleja el bajo impacto (overhead) en el ancho de banda que posee el protocolo XMPP al estructurar la data en bloques pequeños.

Un hallazgo de gran valor se presenta en la trama 980 de la figura 5, donde el analizador detecta un paquete de [TCP Retransmission] proveniente del servidor (172.19.0.2) hacia el puerto 51052 del cliente, seguido de un ACK duplicado con banderas `SLE` y `SRE`. Este fenómeno ocurre comúnmente en entornos de red virtualizados mediante contenedores cuando hay pequeñas ráfagas de procesamiento por parte del demonio de ejabberd al gestionar colas de mensajes concurrentes, lo que obligó a la pila TCP del contenedor a retransmitir un segmento de 48 bytes para asegurar que la sesión persistente no sufriera desconexiones o pérdidas de sincronía. Confirmando la resiliencia del canal de transporte TCP frente a micro-latencias del puente virtual.

Suposiciones sobre Modificación de Tráfico (On-the-fly)

Al analizar la estructura de XMPP, se levantan las siguientes hipótesis en caso de que un atacante logre interceptar y modificar los paquetes *on-the-fly* (por ejemplo, mediante un ataque de *Downgrade* o *SSL Stripping* que permita leer el XML en texto plano):

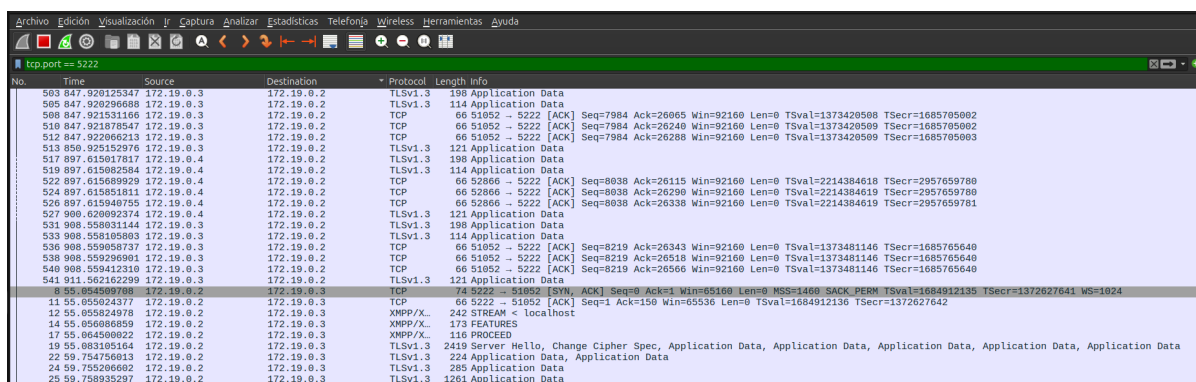
- **Modificación de mensajes:** Si un atacante intercepta un paquete en tránsito y altera el texto dentro del nodo `<body>`, el servidor Ejabberd enrutará el paquete modificado sin emitir alertas. El cliente Profanity receptor procesará el XML válido y renderizará el mensaje falso en la terminal del usuario. A nivel de funcionamiento, ninguno de los softwares colapsará (no habrá *crash*), ya que XMPP asume que la carga útil es correcta si el XML está bien formado. La repercusión afecta netamente a la integridad de la información en la capa de aplicación.
- **Alteración de presencia:** Si se intercepta una estrofa de presencia y se modifica su atributo en el aire (por ejemplo, inyectando un `type='unavailable'`), el servidor procesará este cambio de estado y lo replicará. En la interfaz gráfica/terminal de Profanity de los contactos, el usuario víctima aparecerá repentinamente como "desconectado" (Offline), afectando la usabilidad y la sincronización de estados. Irónicamente, a nivel de red, la sesión TCP subyacente en el puerto 5222 y el túnel XML seguirían completamente activos.
- **Intervención de STARTTLS:** Si se interviene la Fase 2 (Apertura de Flujos) y se borra la etiqueta `<starttls>` de la trama de capacidades del servidor (`<stream:features>`), las repercusiones dependerán de la configuración del software. Si Ejabberd tiene el cifrado configurado como estricto, abortará la sesión en cuanto el cliente intente enviar credenciales en texto plano, causando que Profanity arroje un error fatal de conexión. Si la política es permisiva, el software seguirá funcionando con normalidad, pero la repercusión de seguridad será crítica: el proceso de autenticación SASL y todos los mensajes posteriores viajarán en texto plano y serán visibles en Wireshark.

Conclusiones

A través del análisis realizado, se demostró que XMPP es un protocolo altamente estructurado que depende de capas subyacentes (como TLS) para su seguridad. La visibilidad inicial de las etiquetas XML facilita su estudio, pero también resalta la criticidad de cifrar el túnel de comunicación, dado que la modificación *on-the-fly* de sus *stanzas* comprometería directamente la integridad y confiabilidad del servicio de mensajería. Los objetivos de despliegue y análisis fueron cumplidos exitosamente.

Anexos

Capturas de Wireshark



No.	Time	Source	Destination	Protocol	Length	Info
503	847.920125347	172.19.0.3	172.19.0.2	TLsv1.3	150	Application Data
505	847.920296688	172.19.0.3	172.19.0.2	TLsv1.3	114	Application Data
508	847.921331166	172.19.0.3	172.19.0.2	TCP	66	51052 → 5222 [ACK] Seq=7984 Ack=26065 Win=92160 Len=0 TSval=1373420508 TSecr=1685705002
510	847.921878547	172.19.0.3	172.19.0.2	TCP	66	51052 → 5222 [ACK] Seq=7984 Ack=26240 Win=92160 Len=0 TSval=1373420509 TSecr=1685705002
512	847.922066213	172.19.0.3	172.19.0.2	TCP	66	51052 → 5222 [ACK] Seq=7984 Ack=26288 Win=92160 Len=0 TSval=1373420509 TSecr=1685705003
513	850.925152976	172.19.0.3	172.19.0.2	TLsv1.3	121	Application Data
517	897.615017817	172.19.0.4	172.19.0.2	TLsv1.3	198	Application Data
519	897.615082584	172.19.0.4	172.19.0.2	TLsv1.3	114	Application Data
522	897.615089929	172.19.0.4	172.19.0.2	TCP	66	52866 → 5222 [ACK] Seq=8038 Ack=26115 Win=92160 Len=0 TSval=2214384618 TSecr=2957659780
524	897.615081811	172.19.0.4	172.19.0.2	TCP	66	52866 → 5222 [ACK] Seq=8038 Ack=26290 Win=92160 Len=0 TSval=2214384619 TSecr=2957659780
526	897.615940755	172.19.0.4	172.19.0.2	TCP	66	52866 → 5222 [ACK] Seq=8038 Ack=26338 Win=92160 Len=0 TSval=2214384619 TSecr=2957659781
527	908.620892374	172.19.0.4	172.19.0.2	TLsv1.3	121	Application Data
531	908.558031144	172.19.0.3	172.19.0.2	TLsv1.3	198	Application Data
533	908.558105083	172.19.0.3	172.19.0.2	TLsv1.3	114	Application Data
536	908.559058737	172.19.0.3	172.19.0.2	TCP	66	51052 → 5222 [ACK] Seq=8219 Ack=26343 Win=92160 Len=0 TSval=1373481146 TSecr=1685705640
538	908.559236903	172.19.0.3	172.19.0.2	TCP	66	51052 → 5222 [ACK] Seq=8219 Ack=26518 Win=92160 Len=0 TSval=1373481146 TSecr=1685705640
540	908.559412310	172.19.0.3	172.19.0.2	TCP	66	51052 → 5222 [ACK] Seq=8219 Ack=26566 Win=92160 Len=0 TSval=1373481146 TSecr=1685705640
541	911.562162299	172.19.0.3	172.19.0.2	TLsv1.3	121	Application Data
555	854508766	172.19.0.2	172.19.0.3	TCP	74	5222 → 51052 [RST] Seq=0 Ack=1 Win=65536 Len=0 MSS=1460 SACK_PERM TSval=1684912136 TSecr=1372627642
11	55.055024377	172.19.0.2	172.19.0.3	TCP	66	5222 → 51052 [ACK] Seq=1 Ack=150 Win=65536 Len=0 TSval=1684912136 TSecr=1372627642
12	55.055824978	172.19.0.2	172.19.0.3	XMPP/X.	242	STREAM < localhost
14	55.060808059	172.19.0.2	172.19.0.3	XMPP/X.	173	FEATURES
17	55.064508022	172.19.0.2	172.19.0.3	XMPP/X.	116	PROCEED
19	55.083185164	172.19.0.2	172.19.0.3	TLsv1.3	2419	Server Hello, Change Cipher Spec, Application Data, Application Data, Application Data, Application Data, Application Data
22	59.754756053	172.19.0.2	172.19.0.3	TLsv1.3	224	Application Data, Application Data
24	59.755286602	172.19.0.2	172.19.0.3	TLsv1.3	283	Application Data
25	59.758935297	172.19.0.2	172.19.0.3	TLsv1.3	261	Application Data

Figura 4: Captura de Wireshark mostrando el inicio del Stream XML de XMPP.

No.	Time	Source	Destination	Protocol	Length	Info
1014	1723.2114490	172.19.0.2	172.19.0.3	TLSv1.3	114	Application Data
1015	1723.2114576	172.19.0.3	172.19.0.2	TCP	66	51052 → 5222 [ACK] Seq=12749 Ack=32842 Win=92160 Len=0 TSval=1374295798 TSecr=1686580292
1016	1723.2519237	172.19.0.4	172.19.0.2	TCP	66	52866 → 5222 [ACK] Seq=13027 Ack=32196 Win=92160 Len=0 TSval=2215210253 TSecr=2958485376
1017	1724.1843053	172.19.0.3	172.19.0.2	TLSv1.3	121	Application Data
1018	1724.2249063	172.19.0.2	172.19.0.3	TCP	66	5222 → 51052 [ACK] Seq=32842 Ack=12804 Win=69632 Len=0 TSval=1686581306 TSecr=1374296771
1019	1737.7624812	172.19.0.3	172.19.0.2	TLSv1.3	273	Application Data
1020	1737.7625323	172.19.0.2	172.19.0.3	TCP	66	5222 → 51052 [ACK] Seq=32842 Ack=13011 Win=69632 Len=0 TSval=1686594843 TSecr=1374310349
1021	1737.7625727	172.19.0.3	172.19.0.2	TLSv1.3	114	Application Data
1022	1737.7625813	172.19.0.2	172.19.0.3	TCP	66	5222 → 51052 [ACK] Seq=32842 Ack=13059 Win=69632 Len=0 TSval=1686594843 TSecr=1374310349
1023	1737.7630842	172.19.0.2	172.19.0.3	TLSv1.3	121	Application Data
1024	1737.7636221	172.19.0.3	172.19.0.2	TCP	66	51052 → 5222 [ACK] Seq=13059 Ack=32897 Win=92160 Len=0 TSval=1374310350 TSecr=1686594844
1025	1737.7638081	172.19.0.2	172.19.0.4	TLSv1.3	325	Application Data
1026	1737.7638154	172.19.0.4	172.19.0.2	TCP	66	52866 → 5222 [ACK] Seq=13027 Ack=32365 Win=92160 Len=0 TSval=2215224766 TSecr=2958499928
1027	1737.7639201	172.19.0.2	172.19.0.4	TLSv1.3	114	Application Data
1028	1737.7639264	172.19.0.4	172.19.0.2	TCP	66	52866 → 5222 [ACK] Seq=13027 Ack=32413 Win=92160 Len=0 TSval=2215224767 TSecr=2958499929
1029	1739.7230959	172.19.0.4	172.19.0.2	TLSv1.3	121	Application Data
1030	1739.7639399	172.19.0.2	172.19.0.4	TCP	66	5222 → 52866 [ACK] Seq=32413 Ack=13082 Win=69632 Len=0 TSval=2958501929 TSecr=2215226726
1033	1745.2544316	172.19.0.4	172.19.0.2	TLSv1.3	198	Application Data
1034	1745.2544839	172.19.0.2	172.19.0.4	TCP	66	5222 → 52866 [ACK] Seq=32413 Ack=13214 Win=69632 Len=0 TSval=2958507419 TSecr=2215232257
1035	1745.2545342	172.19.0.4	172.19.0.2	TLSv1.3	114	Application Data
1036	1745.2545463	172.19.0.2	172.19.0.4	TCP	66	5222 → 52866 [ACK] Seq=32413 Ack=13262 Win=69632 Len=0 TSval=2958507419 TSecr=2215232257
1037	1745.2553649	172.19.0.2	172.19.0.4	TLSv1.3	121	Application Data
1038	1745.2554115	172.19.0.4	172.19.0.2	TCP	66	52866 → 5222 [ACK] Seq=13262 Ack=32468 Win=92160 Len=0 TSval=2215232258 TSecr=2958507420
1039	1745.2555796	172.19.0.2	172.19.0.4	TLSv1.3	241	Application Data
1040	1745.2556355	172.19.0.2	172.19.0.4	TLSv1.3	114	Application Data
1041	1745.2556364	172.19.0.4	172.19.0.2	TCP	66	52866 → 5222 [ACK] Seq=13262 Ack=32643 Win=92160 Len=0 TSval=2215232258 TSecr=2958507420
1042	1745.2556445	172.19.0.4	172.19.0.2	TCP	66	52866 → 5222 [ACK] Seq=13262 Ack=32691 Win=92160 Len=0 TSval=2215232258 TSecr=2958507420
1043	1747.8981249	172.19.0.4	172.19.0.2	TLSv1.3	121	Application Data
1044	1748.9380527	172.19.0.2	172.19.0.4	TCP	66	5222 → 52866 [ACK] Seq=32691 Ack=13317 Win=69632 Len=0 TSval=2958510196 TSecr=2215234993
1047	1756.1010802	172.19.0.3	172.19.0.2	TLSv1.3	198	Application Data
1048	1756.1026394	172.19.0.2	172.19.0.3	TLSv1.3	241	Application Data
1049	1756.1026971	172.19.0.3	172.19.0.2	TLSv1.3	114	Application Data
1050	1756.1030363	172.19.0.2	172.19.0.3	TLSv1.3	114	Application Data
1051	1756.1033047	172.19.0.2	172.19.0.3	TLSv1.3	121	Application Data
1054	1756.1070523	172.19.0.3	172.19.0.2	TCP	78	51052 → 5222 [ACK] Seq=13239 Ack=33175 Win=92160 Len=0 TSval=1374328694 TSecr=1686613184 SLE=33126 SRE=33175
1054	1756.1060755	172.19.0.3	172.19.0.2	TLSv1.3	121	Application Data

Figura 5: Captura de Wireshark mostrando el intercambio de mensajes.